

Basics of Coding Theory in Data Compression

Introduction

Coding theory deals with how information (data) can be represented efficiently and reliably. In **data compression**, we focus on source coding (reducing redundancy in data representation). Goal: Represent data with the **minimum number of bits** while still being able to **perfectly reconstruct** (lossless) or approximately reconstruct (lossy).

Types of Coding in Data Compression:

1. Source Coding (main focus here):

- Removes redundancy from the source.
- Example: Huffman coding, Arithmetic coding, LZW.
- **Text compression:** ZIP, GZIP
- **Image compression:** PNG, JPEG

2. Channel Coding:

- Adds redundancy for error detection/correction during transmission.
- Example: Hamming code, Reed-Solomon (used in CDs, QR codes), LDPC.

Mathematical Preliminaries

1. Alphabet & Symbols

- Source emits symbols from an alphabet $\mathcal{A}$.
- Example: $\mathcal{A} = \{A, B, C, D\}$.

2. Probability Distribution

- Each symbol has probability $p(x)$.
- Example: $P(A) = 0.5, P(B) = 0.25, P(C) = 0.15, P(D) = 0.10$.

3. Codeword

- Binary string assigned to each symbol.
- Example: $A \rightarrow 0, B \rightarrow 10, C \rightarrow 110, D \rightarrow 111$.

4. Code Length

- If symbol x is mapped to a codeword of length $l(x)$, then average code length is:

$$L_{avg} = \sum_{x \in \mathcal{A}} p(x) \cdot l(x)$$

5. Entropy

- The theoretical lower bound on average code length:

$$H(X) = - \sum_{x \in \mathcal{A}} p(x) \log_2 p(x)$$

- If $L_{avg} \approx H(X)$, compression is optimal.

Desirable Properties of Codes

1. Uniquely Decodable

- Every encoded message must map to exactly one original message.
- Example: If $A \rightarrow 0, B \rightarrow 01$, then "01" is ambiguous (could be B or A+A).

2. Prefix-Free (Instantaneous) Codes

- No codeword is the prefix of another.
- Example: $A \rightarrow 0, B \rightarrow 10, C \rightarrow 110, D \rightarrow 111$.
- Makes decoding easy.

3. Efficiency

- Closer L_{avg} is to $H(X)$, the more efficient.

Kraft–McMillan Inequality:

Condition for the existence of a prefix-free code:

$$\sum_{i=1}^n 2^{-l_i} \leq 1$$

Example: Code lengths {1,2,3,3}

$$2^{-1} + 2^{-2} + 2^{-3} + 2^{-3} = 0.5 + 0.25 + 0.125 + 0.125 = 1$$

Valid prefix-free code possible.

Key Coding Techniques in Compression

(a) Shannon–Fano Coding

- Based on probability, recursively splits symbols into groups.
- Not always optimal.

(b) Huffman Coding (Optimal Prefix-Free)

- Algorithm:

- Arrange symbols by probability.
- Merge two least probable symbols into a new node.
- Repeat until one tree remains.
- Assign 0/1 to branches → get codewords.

Used in JPEG, MP3, GZIP.

(c) Arithmetic Coding:

- Encodes an entire sequence into a single fraction in $[0,1)$.
- Higher efficiency for skewed distributions.

(d) Dictionary-Based (Lempel–Ziv: LZ77, LZW)

- Builds a dictionary of repeated patterns.
- Used in **ZIP, PNG, GIF**.

Performance Measures:

1. Average Code Length:

$$L_{avg} = \sum p(x)l(x)$$

2. Entropy (Lower Bound):

$$H(X) = - \sum p(x) \log_2 p(x)$$

3. Efficiency:

$$\eta = \frac{H(X)}{L_{avg}}$$

4. Redundancy:

$$R = 1 - \eta$$

Applications:

File compression: ZIP, RAR, GZIP → use Huffman + LZ.

Images:

- JPEG → Huffman + Quantization + DCT.
- PNG → LZ77 + Huffman.

Videos: MPEG, H.264 → Arithmetic + Run-Length Coding.

Texts & Logs: Storage and transfer with minimum bits.

Networking: Protocol header compression.

Coding theory underpins lossless compression. Entropy gives the lower bound for compression.

Huffman coding = most widely used optimal prefix-free code. Arithmetic coding = more powerful but computationally heavy. Dictionary-based methods = practical for large data (ZIP, PNG).